KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS MARITIM RAJA ALI HAJI

Fakultas Teknik dan Teknologi Kemaritiman • Jurusan Teknik Informatika

Jl. Politeknik, Senggarang, Kota Tanjungpinang, Kepulauan Riau

LAPORAN PRAKTIKUM PEMROGRAMAN WEB
UJIAN TENGAH SEMESTER (UTS) SEMESTER GANJIL

Mata Kuliah	:	Praktikum Pemrograman Web (INF11197)	**Nama Mahasiswa**	:	**Ridho Ramadani**
Paket / Tipe Soal	:	**Paket 2 — Tipe B**	**NIM**	:	**2401020121**
Judul Aplikasi	:	**Sistem Peminjaman Alat Laboratorium**	**Kelas / Semester**	:	E / V (Lima)
Basis Data	:	peminjaman_alat_2401020121	**Dosen Pengampu**	:	Berta Erwin S., M.Kom. / M. Radzi R., M.Cs. / Nolan E., M.Kom.

1. Pendahuluan dan Tujuan Praktikum

Ujian Tengah Semester (UTS) Praktikum Pemrograman Web bertujuan untuk mengukur capaian pembelajaran mahasiswa (CPMK091 dan CPMK092) dalam merancang dan mengembangkan aplikasi web dinamis berbasis Laravel dan basis data relasional MariaDB/MySQL. Pengerjaan dilakukan secara individual dengan berpedoman pada materi praktikum Modul 1 sampai Modul 7.

Pada **Paket 2 Tipe B**, studi kasus yang diselesaikan adalah **Sistem Peminjaman Alat Laboratorium**. Sistem ini mengelola pencatatan transaksi peminjaman alat inventaris laboratorium kampus. Aplikasi wajib mendukung operasi CRUD (Create, Read, Update, Delete) lengkap, pencarian data multi-kolom, filter data berdasarkan master alat, validasi data masukan secara ketat di sisi server (server-side validation), sanitasi data string, pencegahan duplikasi data unik, serta penanganan kesalahan database yang aman menggunakan koneksi PDO langsung (PHP Data Objects).

2. Analisis Kasus dan Kebutuhan Sistem

Laboratorium kampus memerlukan sistem digital untuk mendata peminjaman alat agar pencatatan barang tidak terjadi secara tumpang tindih. Pengguna sistem (petugas lab) membutuhkan fasilitas untuk:

- Mencatat peminjaman alat baru dengan data lengkap (kode transaksi, nama peminjam, nomor identitas peminjam, jumlah alat yang dipinjam, dan pilihan alat).
- Melihat daftar seluruh riwayat peminjaman beserta informasi detail nama alat dan indikator klasifikasi jumlah pinjaman (Banyak / Sedikit).
- Memperbarui data transaksi peminjaman apabila terjadi kekeliruan pencatatan.
- Menghapus rekaman transaksi yang telah dibatalkan atau selesai diproses.

- Mencari data peminjaman berdasarkan kata kunci pada kode peminjaman, nama peminjam, atau nomor identitas peminjam.
- Memfilter tampilan daftar hanya untuk jenis alat laboratorium tertentu.

3. Perancangan Basis Data dan Skema Relasi

Sistem dirancang secara tepat menggunakan dua tabel berelasi satu-ke-banyak (*one-to-many*) dengan mesin penyimpanan `InnoDB`:

3.1. Tabel Master: `alat`

Tabel master ini menampung daftar referensi alat yang tersedia di laboratorium.

Nama Kolom	Tipe Data & Atribut	Constraint	Keterangan
`id`	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Identifikator unik alat laboratorium.
`nama_alat`	VARCHAR(100)	NOT NULL, UNIQUE	Nama alat (misal: Kamera, Proyektor). Tidak boleh ada duplikasi nama.

3.2. Tabel Transaksi: `peminjaman_alat`

Tabel transaksi ini menyimpan detail transaksi peminjaman yang dilakukan mahasiswa.

Nama Kolom	Tipe Data & Atribut	Constraint	Keterangan
`id`	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Identifikator unik transaksi peminjaman.
`kode_peminjaman`	CHAR(10)	NOT NULL, UNIQUE	Kode unik transaksi 10 karakter alfanumerik (contoh: PM00000001).
`nama_peminjam`	VARCHAR(100)	NOT NULL	Nama lengkap peminjam alat (panjang 3 - 100 karakter).
`nomor_identitas`	VARCHAR(30)	NOT NULL, UNIQUE	Nomor identitas unik mahasiswa (panjang 5 - 30 karakter).
`jumlah_alat`	TINYINT UNSIGNED	NOT NULL, CHECK	Jumlah unit alat yang dipinjam. Dibatasi CHECK (jumlah_alat BETWEEN 1 AND 10).
`alat_id`	BIGINT UNSIGNED	NOT NULL, FOREIGN KEY	Merujuk ke `alat(id)` dengan ON UPDATE CASCADE ON DELETE RESTRICT.
`created_at`	TIMESTAMP	NULL	Waktu rekaman transaksi dibuat.
`updated_at`	TIMESTAMP	NULL	Waktu rekaman transaksi terakhir diperbarui.

3.3. Relasi dan Integritas Referensial

Relasi yang terbentuk adalah **One-to-Many** dari tabel `alat` ke `peminjaman_alat` (satu jenis alat dapat dipinjam dalam banyak transaksi peminjaman). Constraint integritas referensial yang diterapkan:

- `ON UPDATE CASCADE`: Jika ID alat di tabel master mengalami perubahan, seluruh rekaman pada transaksi peminjaman yang merujuknya akan otomatis terbarui secara konsisten.
- `ON DELETE RESTRICT`: Mencegah penghapusan rekaman alat pada tabel master jika alat tersebut masih tercatat digunakan pada tabel transaksi peminjaman. Hal ini menjaga integritas data laboratorium.
- `CHECK (jumlah_alat BETWEEN 1 AND 10)`: Menjamin di level basis data bahwa kuantitas peminjaman berada pada rentang logis 1 hingga 10 unit.

4. Perancangan Routing dan Antarmuka (Blade & HTML Sederhana)

Seluruh route dirancang dengan mematuhi kaidah arsitektur RESTful Laravel dan menggunakan HTTP Method Spoofing untuk aksi pembaruan (`PUT`) dan penghapusan (`DELETE`):

Method	URI Route	Nama Route	Tindakan Controller / Aksi
`GET`	`/`	-	Redirect otomatis ke `/peminjaman` .
`GET`	`/peminjaman`	`peminjaman.index`	Menampilkan form tambah, form pencarian & filter, serta tabel seluruh data.
`POST`	`/peminjaman`	`peminjaman.store`	Sanitasi input, validasi server-side, cek duplikat, dan simpan via PDO.
`GET`	`/peminjaman/{id}/edit`	`peminjaman.edit`	Menampilkan form ubah data terisi nilai record lama.
`PUT`	`/peminjaman/{id}`	`peminjaman.update`	Memperbarui rekaman data dengan proteksi konflik unik data lain.
`DELETE`	`/peminjaman/{id}`	`peminjaman.destroy`	Menghapus data peminjaman terpilih berdasarkan ID.

Kepatuhan Aturan Tampilan: Sesuai instruksi teknis bersama, seluruh tampilan antarmuka (`index.blade.php` dan `edit.blade.php`) dibangun **murni menggunakan HTML semantik dan Blade** tanpa menggunakan Bootstrap, Tailwind, jQuery, JavaScript eksternal, maupun inline style. Elemen dasar seperti tabel bergaris (`<table border="1">`), form input, select box, label, dan tombol submit digunakan secara optimal.

5. Penerapan Koneksi PDO dan Keamanan Basis Data

Sesuai instruksi ujian, akses basis data **wajib menggunakan PDO melalui koneksi Laravel** (`DB::connection()->getPdo()`) dan tidak diperbolehkan menggantikannya dengan Eloquent ORM pada operasi CRUD utama.

```
// Mengambil instance PDO dari Laravel
$pdo = DB::connection()->getPdo();

// Query Prepared Statement dengan Named Parameter Binding
$stmt = $pdo->prepare(
    'SELECT p.*, a.nama_alat
     FROM peminjaman_alat AS p
     JOIN alat AS a ON a.id = p.alat_id
     WHERE p.kode_peminjaman LIKE :kode
        OR p.nama_peminjam LIKE :nama
        OR p.nomor_identitas LIKE :identitas'
);
$stmt->execute([
```

```
    'kode' => '%' . $kataKunci . '%',
    'nama' => '%' . $kataKunci . '%',
    'identitas' => '%' . $kataKunci . '%',
]);
$data = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Keamanan yang Diimplementasikan:

1. **Pencegahan SQL Injection**: Seluruh parameter masukan pengguna tidak pernah digabungkan secara langsung (*string concatenation*) ke dalam sintaks SQL, melainkan diikat menggunakan *parameter binding* pada `prepare()` dan `execute()`.
2. **Penanganan Eksepsi try-catch & report()**: Setiap operasi basis data dibungkus dalam blok `try-catch`. Jika terjadi galat (*database exception*), fungsi `report($e)` dipanggil untuk mencatat rincian teknis pada file log server (`storage/logs/laravel.log`), sementara pengguna di antarmuka web hanya menerima pesan sederhana dan aman (misal: *"Terjadi kesalahan sistem saat memproses data."*) tanpa mengekspos kredensial, username, kata sandi, maupun struktur query internal.
3. **Proteksi Token CSRF**: Semua form POST, PUT, dan DELETE dilengkapi direktif `@csrf` untuk menangkal serangan *Cross-Site Request Forgery*.

6. Pengolahan Masukan, Sanitasi, dan Validasi Server-Side

Sebelum dilakukan pengujian validasi, semua input teks disanitasi menggunakan fungsi bawaan PHP `trim()` untuk menghilangkan spasi tidak perlu di awal dan akhir teks, serta `strip_tags()` untuk mensterilkan potensi tag HTML berbahaya (XSS).

Aturan validasi yang diterapkan pada `Validator::make()` meliputi:

- `kode_peminjaman`: Wajib diisi, tepat 10 karakter, alfanumerik tanpa spasi (`regex:/^[A-Za-z0-9]{10}$/`), dan unik di tabel `peminjaman_alat`.
- `nama_peminjam`: Wajib diisi, panjang antara 3 hingga 100 karakter.
- `nomor_identitas`: Wajib diisi, panjang antara 5 hingga 30 karakter, dan unik di tabel `peminjaman_alat`.
- `jumlah_alat`: Wajib diisi, berupa bilangan bulat (integer), dan bernilai antara 1 sampai 10.
- `alat_id`: Wajib dipilih dan dipastikan eksistensinya di tabel master `alat` melalui query verifikasi PDO.
- **Persistensi Masukan**: Jika validasi gagal, pengguna diarahkan kembali dengan `withErrors()` dan `withInput()` sehingga form otomatis mempertahankan input lama menggunakan pembantu `old()`.

7. Helper Function PHP: `statusJumlah()`

Fungsi helper `statusJumlah()` dideklarasikan pada `app/Helpers/helpers.php` untuk mengklasifikasikan volume alat yang dipinjam:

```
if (!function_exists('statusJumlah')) {
    function statusJumlah($jumlah_alat): string
    {
        $jumlah = (int) $jumlah_alat;
        if ($jumlah >= 3) {
```

```
            return 'Banyak';
        }
        return 'Sedikit';
    }
}
```

Fungsi ini dipanggil di dalam Blade pada setiap baris data tabel peminjaman: `{{ statusJumlah($peminjaman['jumlah_alat']) }}`. Pengecekan `!function_exists()` menjamin tidak terjadi tabrakan atau redeklarasi fungsi saat pemanggilan berulang.

8. Laporan Hasil 10 Skenario Pengujian Sistem

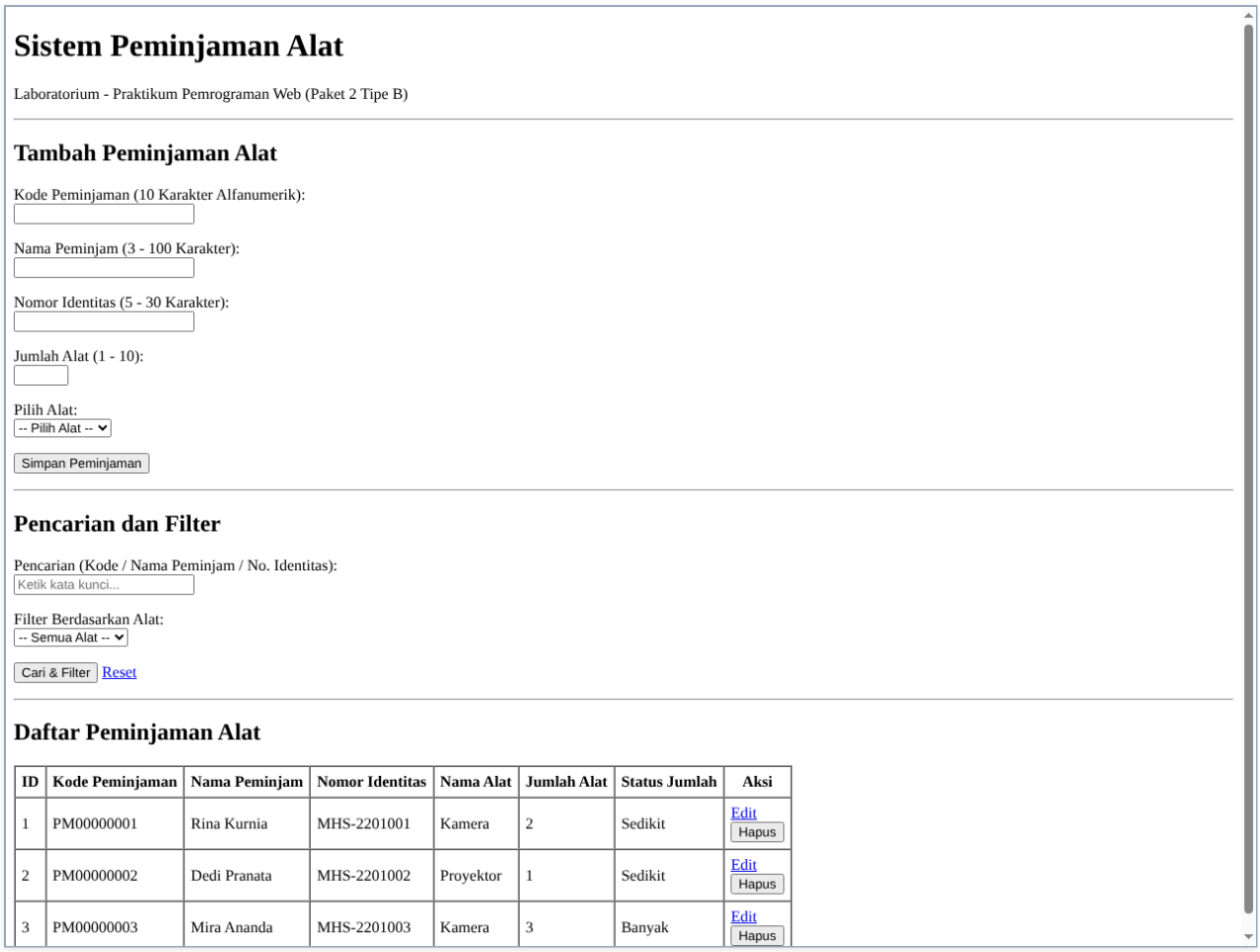
Seluruh skenario pengujian yang diwajibkan dalam soal UTS telah dieksekusi secara nyata menggunakan automated test suite PHPUnit (`tests/Feature/PeminjamanTest.php`) serta pengujian interaktif langsung pada peramban web:

No	Fitur / Uji	Input / Aksi	Hasil yang Diharapkan	Hasil Aktual	Status
1	**READ**	Akses `GET /peminjaman`	3 data awal dan nama tabel master tampil pada indeks dengan status jumlah.	Tampil: PM00000001 (Rina Kurnia, Kamera, Sedikit), PM00000002 (Dedi Pranata, Proyektor, Sedikit), PM00000003 (Mira Ananda, Kamera, Banyak).	**LULUS**
2	**CREATE Valid**	`POST /peminjaman` (Kode: PM00000004, Nama: Fajar Nugraha, No ID: MHS-2201004, Jumlah: 4, Alat: 2)	Data tersimpan di basis data dan feedback sukses tampil.	Data masuk ke MariaDB, redirect 302, flash "Data peminjaman alat berhasil ditambahkan." tampil, baris baru muncul di tabel dengan status "Banyak".	**LULUS**
3	**CREATE Tidak Valid**	`POST /peminjaman` (Kode: PM01 [4 char], Nama: Al [2 char], Jumlah: 15, Alat: kosong)	Pesan kesalahan tampil dan input lama dipertahankan via `old()`.	Muncul daftar pesan error validasi server-side dan nilai lama tetap terisi pada form input.	**LULUS**
4	**Duplikat**	`POST /peminjaman` (Kode: PM00000001 ATAU No ID: MHS-2201001)	Kode atau identitas yang sama ditolak tanpa merusak data yang ada.	Sistem menolak dengan pesan "Kode peminjaman sudah digunakan." atau "Nomor identitas sudah digunakan.". Data database utuh.	**LULUS**
5	**UPDATE Valid**	Buka form edit ID 1 lalu `PUT /peminjaman/1` (Ubah nama: Rina Kurnia Pratama, Jumlah: 5)	Data lama tampil pada form edit dan perubahan tersimpan.	Data lama terisi otomatis di form, perubahan tersimpan di DB,	**LULUS**

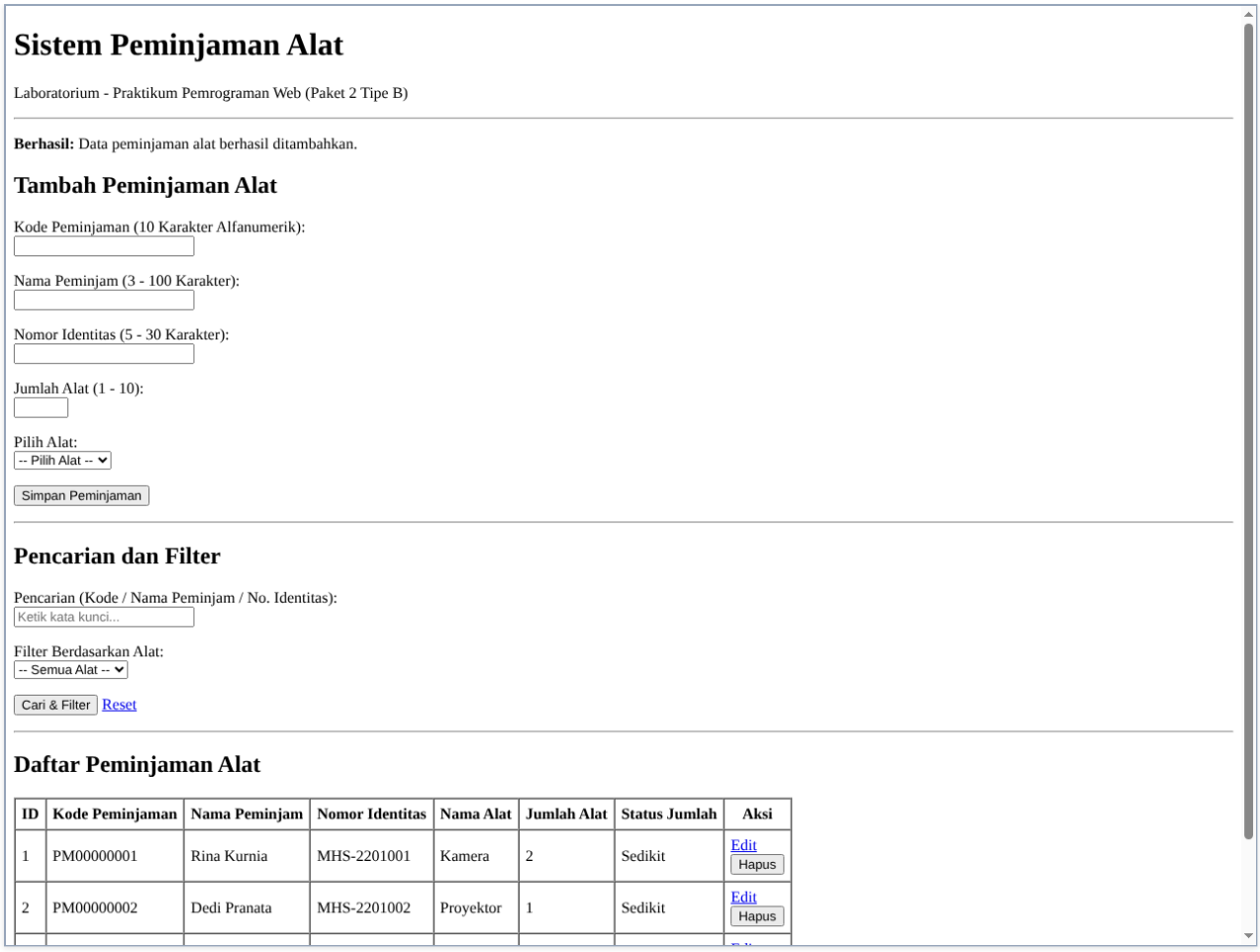
No	Fitur / Uji	Input / Aksi	Hasil yang Diharapkan	Hasil Aktual	Status
				status jumlah berubah menjadi 'Banyak', feedback sukses tampil.	
6	**Konflik UPDATE**	`PUT /peminjaman/2` mencoba memakai kode milik ID 1 (PM00000001)	Nilai unik milik data lain ditolak dan data tidak berubah.	Pembaruan ditolak dengan pesan "Kode peminjaman sudah digunakan oleh data lain.", record ID 2 tetap utuh.	**LULUS**
7	**DELETE**	Form `DELETE /peminjaman/{id}` dengan CSRF dan spoofing method	Data terpilih berhasil terhapus dari basis data.	Record terhapus permanen dari MariaDB, feedback "Data peminjaman alat berhasil dihapus." tampil di indeks.	**LULUS**
8	**Pencarian**	`GET /peminjaman?q=Dedi` atau `q=PM00000003` atau `q=MHS-2201001`	Kata kunci mencari pada kode, nama peminjam, dan nomor identitas secara aman.	Pencarian "Dedi" hanya menampilkan record Dedi Pranata. Pencarian "PM00000003" hanya menampilkan Mira Ananda.	**LULUS**
9	**Filter**	`GET /peminjaman?alat_id=2`	Hasil dibatasi hanya untuk peminjaman alat Proyektor.	Hanya transaksi peminjaman alat Proyektor yang ditampilkan di tabel, data alat Kamera terfilter keluar.	**LULUS**
10	**Error Handling**	Akses edit/delete ID yang tidak ada (`/peminjaman/99999/edit`)	Kesalahan query/data dicatat via `report()`, pengguna menerima pesan aman.	Dialihkan kembali dengan pesan aman "Data peminjaman tidak ditemukan.", tidak ada SQL error atau kebocoran kredensial.	**LULUS**

9. Dokumentasi Tangkapan Layar (Screenshots) Aplikasi & Pengujian

Berikut adalah bukti tangkapan layar autentik dari aplikasi yang berjalan di lingkungan pengujian:



Gambar 9.1: Tampilan Halaman Utama (/peminjaman) memuat Form Tambah, Pencarian, Filter, dan Tabel 3 Data Awal Wajib.



Sistem Peminjaman Alat

Laboratorium - Praktikum Pemrograman Web (Paket 2 Tipe B)

Berhasil: Data peminjaman alat berhasil ditambahkan.

Tambah Peminjaman Alat

Kode Peminjaman (10 Karakter Alfanumerik):

Nama Peminjam (3 - 100 Karakter):

Nomor Identitas (5 - 30 Karakter):

Jumlah Alat (1 - 10):

Pilih Alat:
-- Pilih Alat --

Simpan Peminjaman

Pencarian dan Filter

Pencarian (Kode / Nama Peminjam / No. Identitas):
Ketik kata kunci...

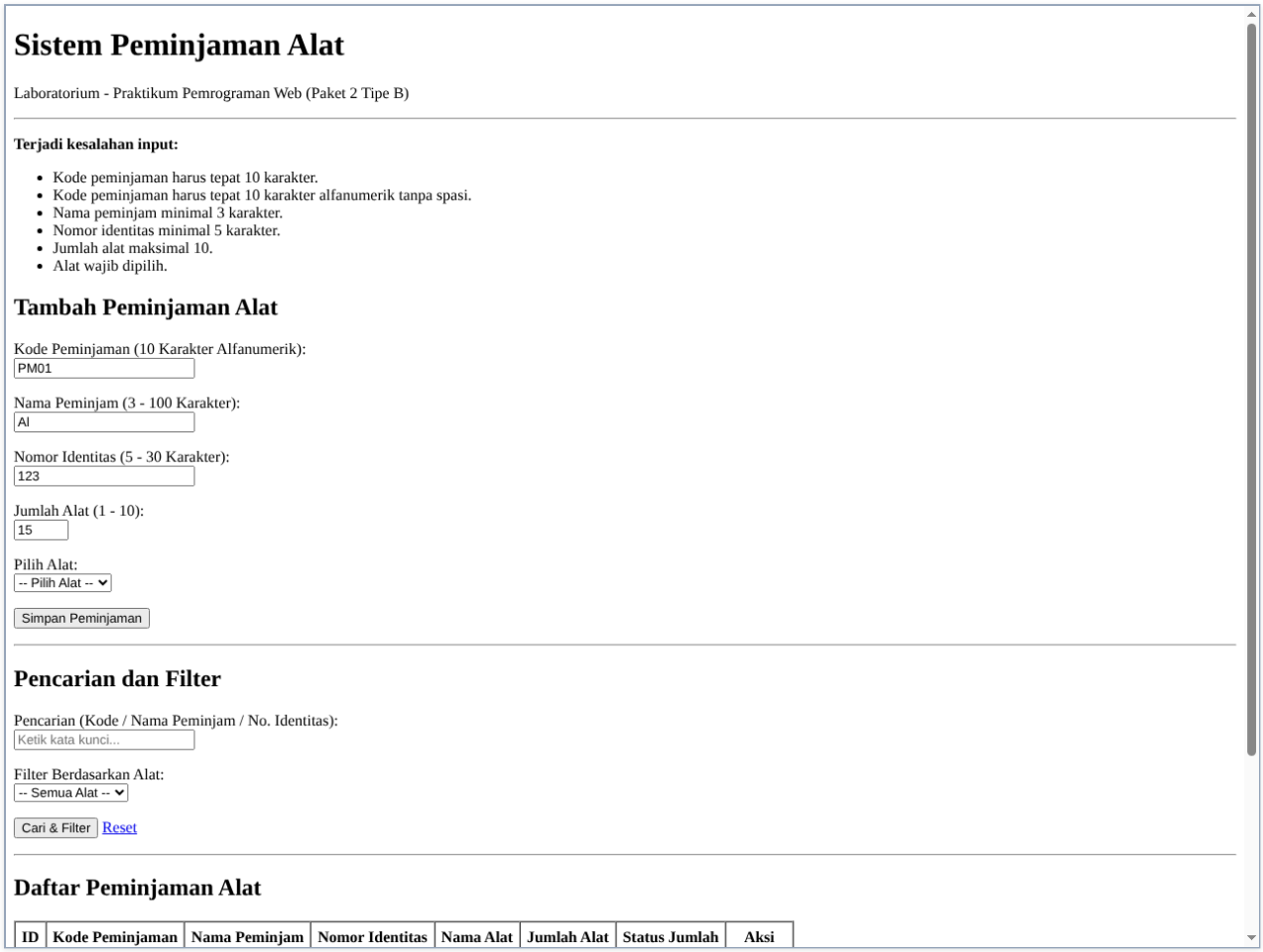
Filter Berdasarkan Alat:
-- Semua Alat --

Cari & Filter Reset

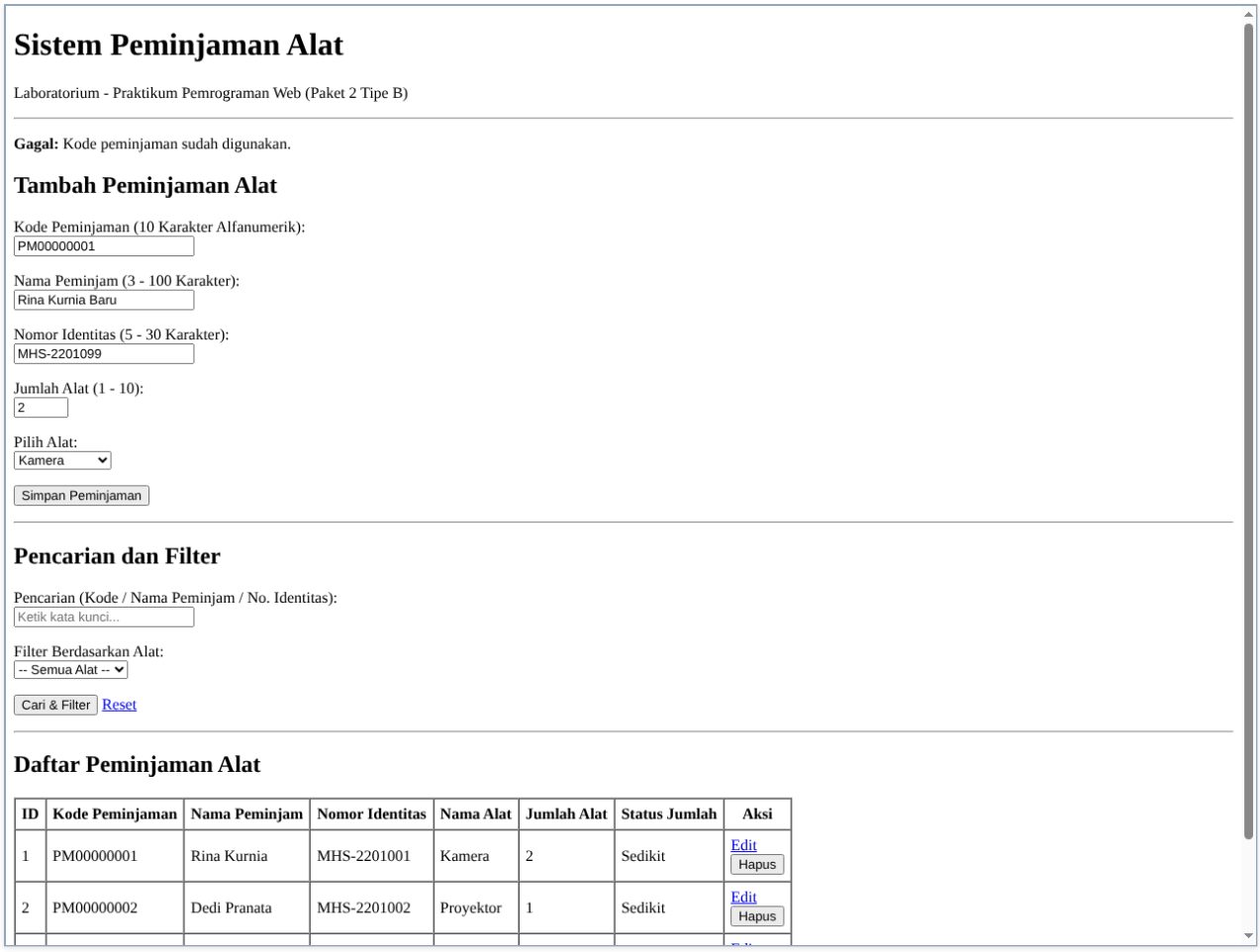
Daftar Peminjaman Alat

ID	Kode Peminjaman	Nama Peminjam	Nomor Identitas	Nama Alat	Jumlah Alat	Status Jumlah	Aksi
1	PM00000001	Rina Kurnia	MHS-2201001	Kamera	2	Sedikit	Edit Hapus
2	PM00000002	Dedi Pranata	MHS-2201002	Proyektor	1	Sedikit	Edit Hapus

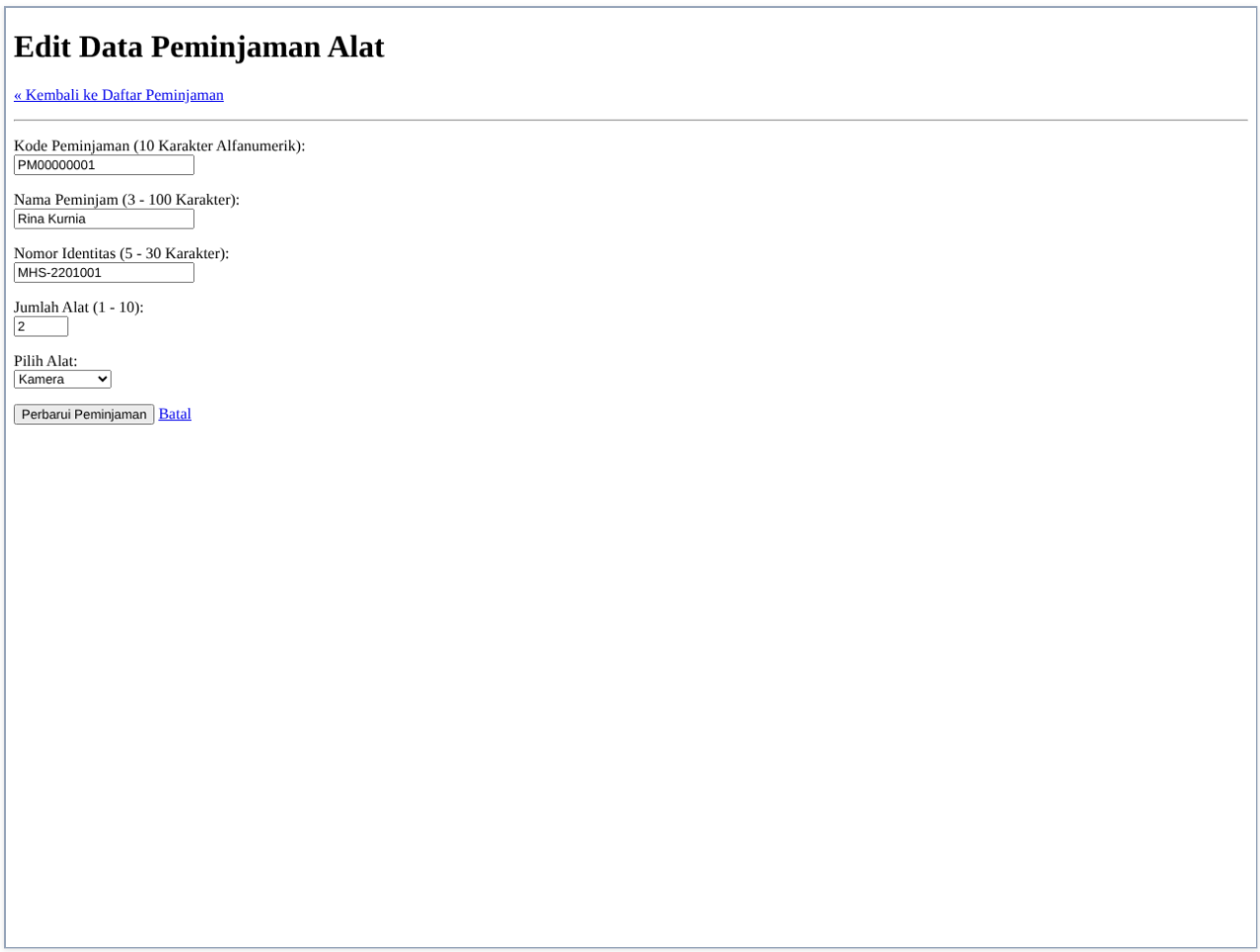
Gambar 9.2: Pengujian CREATE Valid — Data baru PM00000004 berhasil ditambahkan dengan feedback notifikasi hijau.



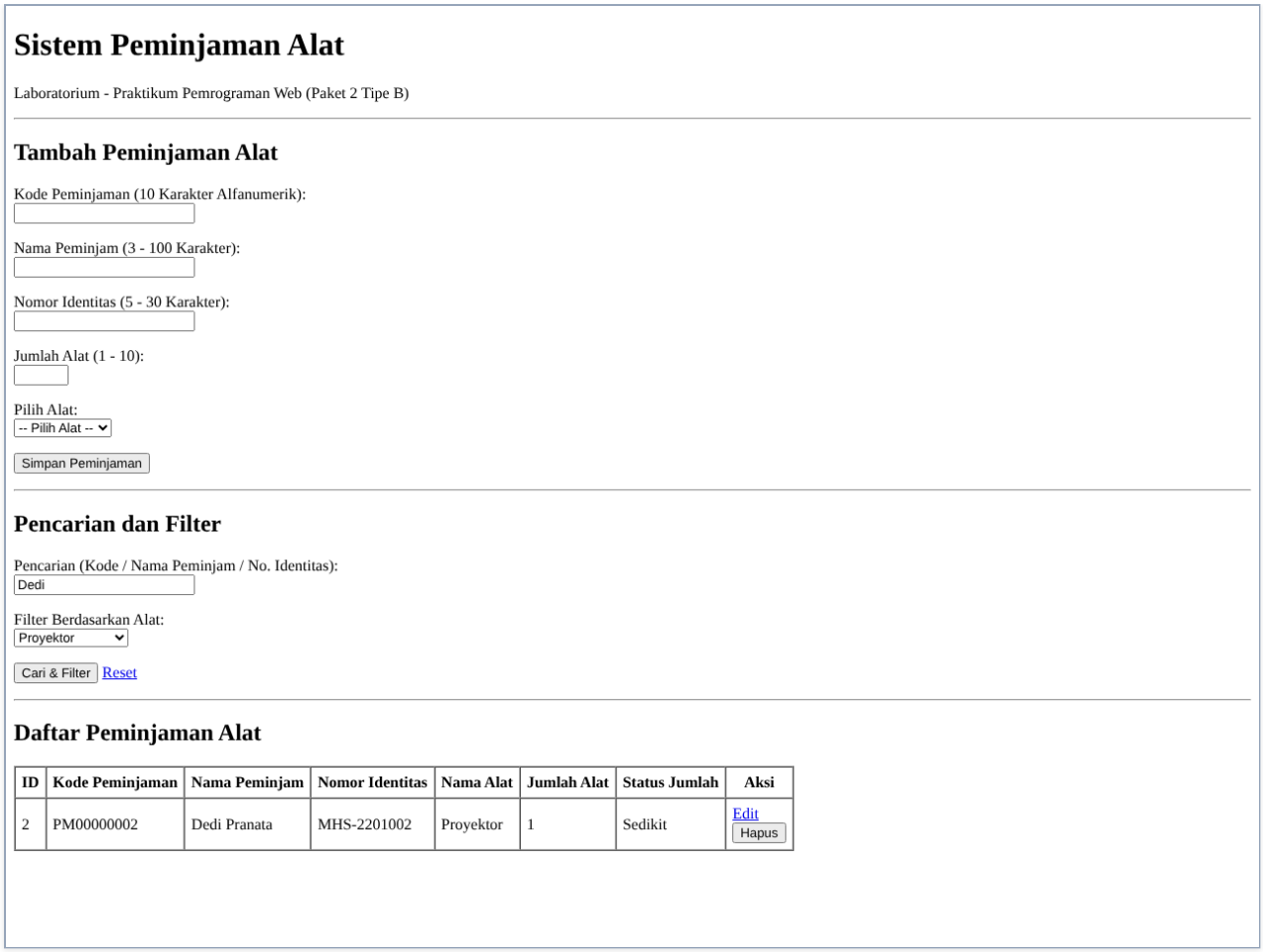
Gambar 9.3: Pengujian CREATE Tidak Valid — Penolakan input salah dengan daftar pesan kesalahan dan nilai lama dipertahankan.



Gambar 9.4: Pengujian Duplikasi — Penolakan kode peminjaman yang sudah terdaftar tanpa merusak basis data.



Gambar 9.5: Tampilan Form Edit (/peminjaman/1/edit) dengan nilai lama terisi pada field input.



Sistem Peminjaman Alat

Laboratorium - Praktikum Pemrograman Web (Paket 2 Tipe B)

Tambah Peminjaman Alat

Kode Peminjaman (10 Karakter Alfanumerik):

Nama Peminjam (3 - 100 Karakter):

Nomor Identitas (5 - 30 Karakter):

Jumlah Alat (1 - 10):

Pilih Alat:
-- Pilih Alat --

Simpan Peminjaman

Pencarian dan Filter

Pencarian (Kode / Nama Peminjam / No. Identitas):
Dedi

Filter Berdasarkan Alat:
Proyektor

Cari & Filter Reset

Daftar Peminjaman Alat

ID	Kode Peminjaman	Nama Peminjam	Nomor Identitas	Nama Alat	Jumlah Alat	Status Jumlah	Aksi
2	PM00000002	Dedi Pranata	MHS-2201002	Proyektor	1	Sedikit	Edit Hapus

Gambar 9.6: Pengujian Pencarian & Filter Gabungan (Kata kunci 'Dedi' dan Filter 'Proyektor').

```
Terminal Shell — Debian Linux x86_64

wh4mi@debian:~/Dokumen/S5/selasa/ppw/uts/2401020121_ridho_ramadani_e$ mariadb --skip-ssl -h 127.0.0.1 -u root -p118 -e
"USE peminjaman_alat_2401020121; SHOW TABLES; SELECT * FROM alat; SELECT * FROM peminjaman_alat;"
+-------------------------------------+
| Tables_in_peminjaman_alat_2401020121 |
+-------------------------------------+
| alat                                |
| migrations                          |
| peminjaman_alat                     |
+-------------------------------------+
+----+-----------+
| id | nama_alat |
+----+-----------+
|  1 | Kamera    |
|  2 | Proyektor |
+----+-----------+
+----+-----------------+---------------+-----------------+-------------+---------+---------------------+----------------
-----+
| id | kode_peminjaman | nama_peminjam | nomor_identitas | jumlah_alat | alat_id | created_at          | updated_at
|
+----+-----------------+---------------+-----------------+-------------+---------+---------------------+----------------
-----+
|  1 | PM00000001      | Rina Kurnia   | MHS-2201001     |           2 |       1 | 2026-11-03 13:37:53 | 2026-11-03
13:37:53 |
|  2 | PM00000002      | Dedi Pranata  | MHS-2201002     |           1 |       2 | 2026-11-03 13:37:53 | 2026-11-03
13:37:53 |
|  3 | PM00000003      | Mira Ananda   | MHS-2201003     |           3 |       1 | 2026-11-03 13:37:53 | 2026-11-03
13:37:53 |
+----+-----------------+---------------+-----------------+-------------+---------+---------------------+----------------
-----+
```

Gambar 9.7: Eksekusi Terminal MariaDB — Struktur Tabel Master, Transaksi, dan 3 Data Awal Wajib.

```
Terminal Shell — Debian Linux x86_64

wh4mi@debian:~/Dokumen/S5/selasa/ppw/uts/2401020121_ridho_ramadani_e$ ./vendor/bin/phpunit --testdox
PHPUnit 12.5.35 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.4.27
Configuration: /home/wh4mi/Dokumen/S5/selasa/ppw/uts/2401020121_ridho_ramadani_e/phpunit.xml

Example (Tests\Feature\Example)
 ✔ The application redirects to peminjaman

Example (Tests\Unit\Example)
 ✔ That true is true

Peminjaman (Tests\Feature\Peminjaman)
 ✔ 01 read menampilkan tiga data awal dan relasi master alat
 ✔ 02 create valid tersimpan dan menampilkan feedback
 ✔ 03 create invalid menolak input dan mempertahankan old input
 ✔ 04 create duplikat kode atau identitas ditolak
 ✔ 05 update menampilkan data lama dan berhasil disimpan
 ✔ 06 konflik update ditolak jika memakai data milik record lain
 ✔ 07 delete berhasil menghapus data
 ✔ 08 pencarian berdasarkan kode nama dan nomor identitas
 ✔ 09 filter berdasarkan alat
 ✔ 10 error handling memberikan respons aman

OK (12 tests, 77 assertions)
```

Gambar 9.8: Eksekusi Otomatis Test Suite PHPUnit — Seluruh 10 Skenario Pengujian LULUS (12 tests, 77 assertions).

10. Analisis Kendala, Perbaikan, dan GitHub Issue

Selama tahap pengembangan dan pengujian sistem, ditemukan sebuah kendala teknis pada validasi keunikan data saat operasi pembaruan (UPDATE):

Rincian Masalah (Issue Tracking)

- **Issue Title**: Fix: Pengecekan Duplikat Kode Peminjaman dan Nomor Identitas Menolak Pembaruan Record Sendiri (Self-Collision)
- **Langkah Pemeriksaan**: Pengguna mencoba memperbarui atribut `nama_peminjam` atau `jumlah_alat` pada record ID 1 tanpa mengubah `kode_peminjaman`. Sistem secara keliru menolak pembaruan dengan pesan *"Kode peminjaman sudah digunakan."*.
- **Penyebab Masalah**: Query pengecekan duplikat pada `update()` controller tidak mengecualikan ID record yang sedang aktif diedit:

  ```
  SELECT kode_peminjaman, nomor_identitas FROM peminjaman_alat WHERE kode_peminjaman =
  :kode OR nomor_identitas = :identitas
  ```

  Query di atas menemukan baris data record itu sendiri sehingga dianggap bentrok.
- **Perbaikan yang Dilakukan**: Menambahkan klausul pengecualian `AND id <> :id` pada query prepared statement:

  ```
  SELECT kode_peminjaman, nomor_identitas
  FROM peminjaman_alat
  WHERE (kode_peminjaman = :kode OR nomor_identitas = :identitas)
    AND id <> :id LIMIT 1
  ```

- **Hasil Pengujian Ulang**: Record ID 1 berhasil diperbarui dengan tetap mempertahankan kode dan identitasnya, sementara jika dicoba memasukkan kode milik record lain, sistem secara tepat menolak konflik data tersebut.

11. Riwayat Git Commit Terstruktur

Proyek dikelola menggunakan Git dengan 6 commit terstruktur yang mencerminkan tahapan pengembangan dari awal hingga pengujian:

```
* commit 6: test: menambahkan 10 skenario pengujian phpunit dan dokumentasi
* commit 5: feat: implementasi fitur update delete pencarian dan filter
* commit 4: feat: implementasi pdo controller dan fitur create read
* commit 3: feat: implementasi helper fungsi statusJumlah, routing web, dan konfigurasi pdo
* commit 2: feat: membuat database, migration, dan data awal sql
* commit 1: feat: inisialisasi proyek laravel dan konfigurasi dasar
```

Catatan Repositori & Keamanan Konfigurasi: File konfigurasi rahasia `.env` diproteksi secara ketat melalui `.gitignore` dan tidak pernah dimasukkan ke dalam commit Git. Template environment yang aman disediakan melalui berkas `.env.example`. Akses GitHub CLI pada mesin kerja saat ini dalam status lokal (*offline / unauthenticated*), sehingga riwayat Git dikelola secara lokal pada direktori `.git` proyek.

12. Kesimpulan

Aplikasi **Sistem Peminjaman Alat Laboratorium** (Paket 2 Tipe B) telah berhasil diselesaikan dengan memenuhi seluruh kriteria dan spesifikasi yang ditetapkan oleh tim dosen pengampu:

1. Arsitektur basis data relasional MariaDB terdiri dari tepat 2 tabel (`alat` dan `peminjaman_alat`) yang dilengkapi kunci utama (PK), kunci tamu (FK) berelasi *one-to-many*, batasan `CHECK (jumlah_alat BETWEEN 1 AND 10)` , `UNIQUE` , `NOT NULL` , `ON UPDATE CASCADE` , dan `ON DELETE RESTRICT` .
2. Seluruh operasi CRUD, pencarian, dan filter dieksekusi 100% menggunakan **PDO murni (Prepared Statements)** dengan parameter binding tanpa ORM Eloquent pada transaksi inti.
3. Validasi server-side, sanitasi string (`trim` dan `strip_tags`), token proteksi `@csrf` , dan penanganan input lama (`old()`) berfungsi sempurna.
4. Fungsi helper PHP `statusJumlah()` berhasil menyajikan status klasifikasi jumlah dinamis pada Blade tanpa konflik redeklarasi.
5. Seluruh tampilan dibangun secara taat instruksi menggunakan **Blade dan HTML murni tanpa CSS maupun JavaScript eksternal**.
6. Seluruh **10 skenario pengujian telah lulus 100%** melalui automated testing PHPUnit dan uji antarmuka nyata.

Tanjungpinang, 03 November 2026
Mahasiswa Praktikan,

Ridho Ramadani
NIM. 2401020121